

1. Fonctions d'Agrégation (Calculs de groupe)

Ces fonctions prennent plusieurs lignes en entrée et retournent **une seule valeur**.

Règle d'or : Ces fonctions ignorent les valeurs NULL (sauf COUNT(*)).

Fonction	Description	Particularité	Exemple
COUNT(*)	Compte le nombre total de tuples (lignes).	Compte TOUT, même les NULL.	SELECT COUNT(*) FROM vin; <i>Résultat : 5</i>
COUNT(colonne)	Compte les valeurs non-nulles d'une colonne.	Ignore les NULL.	SELECT COUNT(qualite) FROM vin;
COUNT(DISTINCT col)	Compte les valeurs <i>uniques</i> non-nulles.	Évite les doublons.	SELECT COUNT(DISTINCT millesime) FROM vin;
AVG(colonne)	Moyenne arithmétique.	Numérique seulement. Ignore NULL.	SELECT AVG(degre) FROM vin;
SUM(colonne)	Somme des valeurs.	Numérique seulement. Ignore NULL.	SELECT SUM(montant) FROM vente;
MIN(colonne)	Valeur minimale.	Marche pour nombres, textes, dates.	SELECT MIN(millesime) FROM vin;
MAX(colonne)	Valeur maximale.	Marche pour nombres, textes, dates.	SELECT MAX(degre) FROM vin;

2. Groupement des données (GROUP BY)

Permet de créer des sous-ensembles de données partageant une valeur commune (ex: par région, par magasin) et d'appliquer une fonction d'agrégation sur chaque groupe.

Syntaxe stricte :

SQL

SELECT colonne_groupe, FONCTION(autre_colonne)

FROM table

GROUP BY colonne_groupe;

Erreurs fréquentes (Sois vigilant ici) :

1. **Règle de projection** : Tout attribut dans le SELECT qui n'est **pas** dans une fonction d'agrégation (SUM, AVG...) **DOIT** figurer dans le GROUP BY.
 - o *Erreur* : SELECT code_postal, nom, AVG(salaire) ... GROUP BY code_postal
-> Plante car "nom" n'est pas groupé.
2. **Pas d'agrégation dans le WHERE** : Le WHERE filtre les lignes *avant* le groupement. On ne peut pas dire WHERE MAX(salaire) > 2000.

Exemple : Afficher le degré minimum par cru.

SQL

SELECT cru, MIN(degre)

FROM vin

WHERE millesime > 1980

GROUP BY cru;

[cite: 304]

3. Filtrage des groupes (HAVING)

Le HAVING est au GROUP BY ce que le WHERE est au SELECT. Il sert à filtrer les groupes **après** que les calculs d'agrégation ont été faits.

- **Différence clé** : WHERE filtre les lignes individuelles. HAVING filtre le résultat d'une fonction de groupe (SUM, COUNT, etc.).

Exemple : Afficher les magasins dont le total des ventes dépasse 1500\$.

SQL

SELECT magasin, SUM(montant)

FROM vente

GROUP BY magasin

HAVING SUM(montant) >= 1500;

[cite: 452, 458]

4. Opérateurs Ensemblistes

Permettent de combiner verticalement les résultats de deux requêtes distinctes.

Condition impérative : Les deux requêtes doivent avoir le **même nombre de colonnes** et des **types compatibles**.

Opérateur	Description	Gestion des doublons
UNION	$A + B$ (Additionne les résultats).	Supprime les doublons.
UNION ALL	$A + B$.	Garde les doublons (plus rapide).
INTERSECT	$A \cap B$ (Seulement les lignes présentes dans les deux).	Supprime les doublons.
MINUS (ou EXCEPT)	$A - B$ (Lignes de A qui ne sont pas dans B).	Supprime les doublons.

Exemple (UNION) : Lister toutes les dates où il y a eu une vente magasin OU une vente internet.

SQL

```
SELECT date FROM vente
```

```
UNION
```

```
SELECT date FROM vente_par_internet;
```

[cite: 490]

5. Sous-requêtes (Subqueries)

On utilise le résultat d'une requête interne (fille) pour filtrer la requête principale (père).

A. Comparaison Unique ($=$, $<$, $>$)

La sous-requête doit renvoyer **exactement une ligne et une colonne**.

- *Erreur fréquente :* Si la sous-requête renvoie plusieurs lignes, le SGBD renvoie l'erreur ORA-01427.

Exemple : Trouver les étudiants qui habitent la même localité que l'enseignant n°1.

SQL

```
SELECT nom FROM etudiant
```

```
WHERE localite = (SELECT localite FROM enseignant WHERE idEnseignant = 1);
```

[cite: 649]

B. Comparaison Multiple (ANY, ALL)

Utilisé quand la sous-requête renvoie une liste de valeurs.

- = ANY : Équivalent à IN. Vrai si égal à au moins une valeur.
- > ALL : Vrai si supérieur à **toutes** les valeurs de la liste.
- < ANY : Vrai si inférieur à **au moins une** des valeurs.

Exemple : Vins dont le millésime est inférieur à n'importe quel (au moins un) vin du Beaujolais.

SQL

```
SELECT * FROM vin
```

```
WHERE millesime < ANY (SELECT millesime FROM vin WHERE region LIKE 'Beaujolais%');
```

[cite: 727]

C. Appartenance (IN, NOT IN)

Vérifie si une valeur existe dans la liste retournée par la sous-requête.

Exemple : Vins provenant d'une région qui produit aussi des crus commençant par 'V'.

SQL

```
SELECT * FROM vin
```

```
WHERE region IN (SELECT region FROM vin WHERE cru LIKE 'V%');
```

[cite: 791]

D. Existence (EXISTS, NOT EXISTS)

C'est souvent une **sous-requête corrélée** : la sous-requête dépend d'une valeur de la requête principale. On ne vérifie pas une valeur, mais si la sous-requête retourne **au moins une ligne** (Vrai) ou **aucune** (Faux).

Exemple : Trouver les ventes magasin qui ont une correspondance (même date) dans les ventes internet.

SQL

```
SELECT * FROM vente v
WHERE EXISTS (
    SELECT * FROM vente_par_internet vi
    WHERE v.date = vi.date -- Corrélation ici
);
```

[cite: 837]

6. Logique Conditionnelle (CASE)

Permet de faire du "IF-THEN-ELSE" directement dans le SELECT pour transformer l'affichage des données.

Forme Simple (sur une colonne)

```
SQL
SELECT cru,
CASE qualite
    WHEN 'A' THEN 'Excellent'
    WHEN 'B' THEN 'Bon'
    ELSE 'Moyen'
END AS avis
FROM vin;
```

[cite: 910]

Forme Recherche (Conditions complexes)

Permet d'utiliser des AND, OR, <, >.

Exemple :

```
SQL
SELECT cru,
CASE
    WHEN qualite = 'A' OR qualite = 'B' THEN 'A acheter'
    WHEN qualite = 'C' AND degre < 0.12 THEN 'A goûter'
```

ELSE 'Ne pas acheter'

END AS conseil

FROM vin;

[cite: 942]